

Reviewer Pack — Minimal Ehrhart Gaps and Resolution Proof Width Inequality

Entry 18786df69299 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-05 12:26:55 UTC

Minimal Ehrhart Gaps and Resolution Proof Width Inequality

Entry ID: 18786df69299 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-05 12:26:55 UTC

1. Conjecture

Field A (mathematical branch): Ehrhart Theory **Field B** (complexity object): Resolution Proof Complexity

Statement:

For every Tseitin formula φ_G associated with a d -regular graph G , the minimal Ehrhart gap ($\Delta(\varphi_G)$) of the polynomial associated with φ_G is linearly correlated with its resolution proof width $w(\varphi_G)$, such that $\Delta(\varphi_G) = \Theta(w(\varphi_G))$.

Rationale (proposer's reasoning):

Ehrhart Theory studies the combinatorial properties of integer points in polytopes, which can be seen as a geometric encoding of computational problems. By connecting this theory with the resolution proof complexity, we may uncover new structural insights into the hardness of satisfiability problems. The Ehrhart gap provides a measure of the complexity of a polytope, and its relationship to resolution proof width could reveal a fundamental connection between geometric properties and algorithmic complexity.

Taxonomy category: Ehrhart_Resolution (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 78f294411c2f7f4e

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For each Tseitin formula φ_G , if $\Delta(\varphi_G)$ is within $\pm 20\%$ of $\Theta(w(\varphi_G))$ across all 30 seeds, then support for the conjecture is provided.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	1.00	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	1.00	SAFE	SAFE
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - Minimal Ehrhart Gaps AND Resolution Proof Width - Ehrhart Theory INTEGRATED WITH Resolution Proof Complexity - $\Delta(\phi_G) = \Theta(w(\phi_G))$ AND Tseitin formula

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, $\leq 90s$ wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def generate_d_regular_graph(n, d):
    if n % d != 0:
        raise ValueError("Graph size must be a multiple of the degree")

    graph = [[] for _ in range(n)]
    edges_added = set()

    def add_edge(u, v):
        if (u, v) not in edges_added and (v, u) not in edges_added:
            graph[u].append(v)
            graph[v].append(u)
            edges_added.add((u, v))
            edges_added.add((v, u))

    for i in range(n):
        for j in range(i + 1, n):
            if len(graph[i]) < d and len(graph[j]) < d:
                add_edge(i, j)

    return graph

def tseitin_formula(graph):
    n = len(graph)
    literals = list(range(1, 2 * n + 1))
    clauses = []

    for i in range(n):
        clauses.append([literals[2 * i], literals[2 * i + 1]])
        for j in graph[i]:
            if j < i:
                clauses.append([literals[2 * i], literals[2 * j + 1]])
            else:
                clauses.append([literals[2 * i + 1], literals[2 * j]])
```

```

        continue
    clauses.append([-literals[2 * i], -literals[2 * j + 1]])
    clauses.append([-literals[2 * i + 1], literals[2 * j]])

return clauses

def compute_polynomial(clauses):
    n = len(clauses)
    poly = [0] * (n + 1)
    for clause in clauses:
        term = 1
        for literal in clause:
            if literal > 0:
                term *= (-1) ** (literal % 2)
        poly[len(clause)] += term
    return poly

def compute_ehrhart_gap(poly):
    n = len(poly)
    ehrhart_sum = 0
    for k in range(n + 1):
        ehrhart_sum += poly[k] * math.comb(k, n)
    return abs(ehrhart_sum - (n + 1))

def resolution_proof_width(formula):
    # Placeholder implementation
    return len(formula)

def run_trial(seed: int) -> dict:
    random.seed(seed)

    results = []
    for _ in range(30):
        n = random.choice([5, 10, 15, 20, 30, 40])
        graph = generate_d_regular_graph(n, 2)
        formula = tseitin_formula(graph)
        poly = compute_polynomial(formula)
        ehrhart_gap = compute_ehrhart_gap(poly)
        proof_width = resolution_proof_width(formula)

        results.append((ehrhart_gap, proof_width))

    mean_gap = sum(gap for gap, _ in results) / len(results)
    mean_width = sum(width for _, width in results) / len(results)
    correlation = sum((gap - mean_gap) * (width - mean_width) for gap, width in results) / len(results)

    if correlation == 0:
        return {
            "metric_name": "Correlation",
            "metric_value": correlation,
            "instances_tested": 30,
            "n_max": max(n for _ in range(30)),
            "conjecture_holds": False,
            "counterexample": "mapping_undefined"
        }

```

```

return {
    "metric_name": "Correlation",
    "metric_value": correlation,
    "instances_tested": 30,
    "n_max": max(n for _ in range(30)),
    "conjecture_holds": abs(correlation) > 1.2 * (mean_gap / mean_width),
    "counterexample": ""
}

if __name__ == "__main__":
    import sys
    seeds = [int(arg) for arg in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value) ** 2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    else:
        first_failing_seed = next(seed for seed, r in zip(seeds, results) if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"mapping_undefined\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_7a87101b.py", line 119, in <module>
    result = run_trial(seed)
              ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_7a87101b.py", line 85, in run_trial
    ehrhart_gap = compute_ehrhart_gap(poly)
                  ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_7a87101b.py", line 69, in compute_ehrhart_gap
    ehrhart_sum += poly[k] * math.comb(k, n)
                    ~~~~^
IndexError: list index out of range

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed before producing data, which means it did not complete its execution to provide evidence for or against the conjecture. | next: Re-run the test with proper error handling and ensure that it completes without crashing.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	25193
2	preregistration	ollama_remote	glm4:latest	0	0	9507
3	novelty	ollama_remote	glm4:latest	0	0	21549
4	novelty	ollama_remote	glm4:latest	0	0	13425
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	46677
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11885
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14078
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13148
9	judge	ollama_remote	glm4:latest	0	0	15427

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 170888 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in research/audit/18786df69299.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/18786df69299.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/18786df69299.tar.gz (if generated)